

CPU Operations vs I/O Operations

CPU Operations (Compute-heavy)

- Tasks that require a lot of processing power (like calculations, loops, data compression, etc.).
- ⌚ Can block the event loop (main thread) if they run for too long.
- **Examples:**
 - Hashing (e.g., bcrypt)
 - Image processing
 - Sorting large datasets

I/O Operations (Input/Output)

- Tasks involving reading/writing data to external sources like files, databases, or network.
- ☒ Non-blocking in Node.js (asynchronous by default).
- Handled using **callbacks, promises, or async/await**.
- **Examples:**
 - Reading/writing files
 - Database queries
 - HTTP requests

Node.js and the Event Loop

- Node.js is **single-threaded**, designed for **I/O-bound** tasks.
- For **CPU-bound** tasks:
 - Use `worker_threads` module
 - Offload to external services
 - Use caching to reduce load

Key Differences

Feature	CPU Operations	I/O Operations
Resource Usage	High CPU	External device/network dependent
Blocking	Yes (if heavy computation)	No (in async style)
Best handled by	Worker threads / offloading	Node.js async/non-blocking style

Async I/O in Node.js (Asynchronous Input/Output)

What is Async I/O?

- Async I/O means Node.js **doesn't wait** for input/output operations to finish.
- It starts the task, **moves on**, and handles the result later using:
 - Callbacks

- Promises
- async/await

How it works in Node.js

- Node.js uses the **event loop** + **libuv** to manage async I/O tasks.
- I/O operations are **offloaded** to the system/kernel or thread pool.
- When done, a **callback** or promise is triggered.

Benefits

- 🚦 Non-blocking (Node can handle many tasks at once)
- 🚀 Fast performance for **I/O-bound** apps like:
 - APIs
 - Chat apps
 - File servers

Examples

Using Callback

```
const fs = require('fs');
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

Using Promise

```
const fs = require('fs').promises;
fs.readFile('file.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Using Async/Await

```
const fs = require('fs').promises;
async function readFile() {
  try {
    const data = await fs.readFile('file.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
readFile();
```

Event-Driven Architecture (EDA)

What is it?

- A software design pattern where the **flow of the program is driven by events**.
- Events are **actions or messages** (like user input, HTTP requests, file read, etc.)
- Each event is handled by a corresponding **listener (or handler)**.

How it works:

1. An **event** occurs (e.g., file read complete, user clicks button).
2. That event is **emitted** (broadcasted).
3. A **listener** function is triggered to respond.

In Node.js:

- Node.js is built on **event-driven architecture**.
- It uses the **EventEmitter** class to handle events.

Example in Node.js:

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

myEmitter.on('greet', () => {
  console.log('Hello Afnur!');
});

myEmitter.emit('greet'); // Triggers the 'greet' event
```

Advantages:

-  Non-blocking and **efficient** for I/O-heavy applications
-  **Loose coupling** – components talk through events, not direct calls
-  Great for **real-time apps** (chat, notification systems, etc.)

Disadvantages:

-  Can get hard to debug (too many nested events)
-  Event chain may be hard to track (callback hell if not structured well)

EventEmitter in Node.js

What is EventEmitter?

- A **core module** in Node.js (from **events** module).
- Helps in **handling custom events** using **.on()** and **.emit()** methods.
- Part of the **event-driven architecture** of Node.js.

Basic Usage:

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

// Listener
myEmitter.on('hello', () => {
  console.log('Hello Afnur!');
});

// Emit event
myEmitter.emit('hello');
```

Common Methods:

Method	Description
<code>.on(event, listener)</code>	Register a listener for an event
<code>.emit(event, [args])</code>	Trigger (emit) an event
<code>.once(event, listener)</code>	Listen only once for the event
<code>.removeListener()</code>	Remove a specific listener
<code>.removeAllListeners()</code>	Remove all listeners for an event

Example with Arguments:

```
myEmitter.on('greet', (name) => {
  console.log(`Hello, ${name}`);
});

myEmitter.emit('greet', 'Afnur');
```

Why use EventEmitter?

- To create **custom events** and **decouple** logic.
- Used in **streams, servers, sockets**, etc.